

Java OOP – The Complete Story, From Zero to Interview-Ready

Before We Begin: Why Does OOP Even Exist?

Imagine writing a banking app where every account is just a pile of loose variables – `balance1` , `balance2` , `ownerName1` , `ownerName2` ... It would collapse under its own weight the moment you had 1,000 customers.

Object-Oriented Programming (OOP) solves this by letting you model real-world things – accounts, cars, students – as self-contained units called **objects**. Everything in this guide builds toward four big ideas (the "four pillars"):

Pillar	One-line idea
Encapsulation	Bundle data + behavior, hide the messy internals
Abstraction	Show <i>what</i> something does, hide <i>how</i>
Inheritance	Reuse and extend existing code instead of rewriting it
Polymorphism	One name, many behaviors

Keep this table in your back pocket – every topic below is really just one of these four ideas wearing a different hat.

Part 1: The Foundation – Classes and Objects

Concept

A **class** is a blueprint. An **object** is the actual thing built from that blueprint.

Why do we need it?

You can't build a house from an idea – you need a floor plan first, then you can build many houses from the same plan. Classes let you define a structure once and stamp out as many objects as you need.

Real-life Analogy

The class `Car` is the design on paper. Your actual Honda City in the parking lot is the **object** — it has physical reality, occupies memory, and has its own color, fuel level, and speed.

Java Example

```
class Car {
    String color;    // state
    int speed;       // state

    void accelerate() {    // behavior
        speed += 10;
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();    // object created
        myCar.color = "Red";
        myCar.accelerate();
    }
}
```

Key Points

- An object has three properties: **state** (data), **identity** (its unique memory location), and **behavior** (its methods).
- A class is a *logical* construct; an object is *physical* — it occupies memory.
- `myCar` doesn't hold the object itself — it holds a **reference** (essentially, the memory address) to it.

 **Remember:** `Car b1 = new Car(); Car b2 = b1;` — now `b1` and `b2` point to the same object. Changing data through `b2` also changes what `b1` sees. No copy is made — only the reference is copied.

Common Beginner Mistakes

- Thinking `Car myCar;` creates an object. It doesn't — it only creates a reference. The object is born only when `new` runs.

- Confusing **parameter** (`i` in `square(int i)`) with **argument** (the `100` in `square(100)`).
-

Part 2: Constructors – Setting Up an Object at Birth

Concept

A **constructor** is special code that runs automatically the instant an object is created with `new`, before you can even touch it.

Why do we need it?

Without one, every new object starts as a blank slate, and you'd have to manually set every field, every time. A constructor guarantees an object is born already in a valid state.

Real-life Analogy

When a new student joins a school, admission paperwork automatically assigns them a roll number and class the moment they're enrolled — you don't fill that in separately later.

Java Example

```
class Box {
    double width;
    Box() {                                // no-argument (default) constructor
        width = 10;
    }
    Box(double w) {                        // parameterized constructor (overloaded)
        width = w;
    }
}

Box b1 = new Box();                      // uses Box()
Box b2 = new Box(5.5);                   // uses Box(double)
```

Key Points

- A constructor has **no return type** — not even `void` — because it implicitly "returns" the class type itself.
- Constructors can be **overloaded**, just like normal methods.

- If you don't write any constructor, Java silently supplies a default one for you.
- **Inheritance + constructors:** a subclass constructor always calls its superclass constructor first (automatically, if you don't do it yourself) – because a superclass must finish setting itself up before the subclass builds on top of it.

```
class Base { Base() { System.out.println("Base built"); } }  
class Derived extends Base { Derived() { System.out.println("Derived built"); } }  
// new Derived() prints: "Base built" then "Derived built"
```

 **Gotcha:** If the superclass has *only* a parameterized constructor (no default one), the subclass **must** explicitly call it using `super(args)` – otherwise the code won't compile.

Part 3: The `this` Keyword – Talking About "Yourself"

Concept

`this` is a reference to the current object – the one whose method is currently running.

Why do we need it?

When a parameter and a field share the same name, Java needs a way to say "I mean *my* field, not the parameter."

Java Example

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name;    // this.name = field, name = parameter  
    }  
}
```

Key Points

- `this` always refers to the object on which the current method was invoked.
 - It's commonly used to resolve naming conflicts and to pass the current object to another method.
-

Part 4: Access Control – Who's Allowed to See What

Concept

Access modifiers decide which parts of your program can reach a class member (field or method).

Why do we need it?

Not everything about an object should be touchable from anywhere. A bank account's balance shouldn't be editable by *any* random class – only through controlled methods.

The Four Levels

Access Modifiers →	private	Default (no modifier)	protected	public
Inside same class	✓	✓	✓	✓
Same package class	✗	✓	✓	✓
Same package subclass	✗	✓	✓	✓
Different package class	✗	✗	✗	✓
Different package subclass	✗	✗	✓	✓

(Most restrictive → Least restrictive: *private* → default → *protected* → *public*)

Key Points

- No modifier at all = **default/package-private** access: visible only within the same package.
- *protected* matters only when **inheritance** is involved – it opens a "door" for subclasses in other packages.

🎓 **Interview Question:** Can a subclass in another package access a *protected* member of the superclass?

Only through **its own object (or a subclass of it)** – not through a plain reference to the superclass. In other words:

```
new Derived().display();    // ✓ allowed
new Base().display();       // ✗ not allowed (different package, no inheritance)
```

The reasoning: the subclass has "earned trust" to manage its own inherited state, but it has no business poking at some *other*, unrelated subclass's protected internals.

Part 5: Pillar 1 – Encapsulation

Concept

Encapsulation means bundling data (fields) and the methods that operate on it into one unit (a class), and hiding the internal data from direct outside access.

Why do we need it?

If every field is public, anyone can set a `bankBalance = -9999`, bypassing all your validation rules. Encapsulation forces changes to go through controlled doors – your methods.

Real-life Analogy

A car's engine is **encapsulated** under the hood. You interact with it through the accelerator and brake pedal (public interface) – you never touch the pistons directly.

Java Example

```
class BankAccount {  
    private double balance;           // hidden data  
  
    public void deposit(double amt) { // controlled access  
        if (amt > 0) balance += amt;  
    }  
    public double getBalance() {      // getter  
        return balance;  
    }  
}
```

Key Points

- Achieved using `private` fields + public **getters** and **setters**.
- Solves a problem at the **implementation level** – it's about protecting data.

- Getters/setters let you add validation, logging, or lazy-computation later without breaking code that uses the class.

Common Beginner Mistakes

- Writing a getter and setter for every field "by habit," even ones that should never be exposed. Only expose what's needed.
-

Part 6: Packages – Organizing the Classroom

Concept

A **package** is a folder-like container that groups related classes and prevents naming collisions.

Why do we need it?

Two different libraries might both have a class called `List`. Packages let both exist peacefully – `java.util.List` vs `myapp.utils.List`.

Key Points

- Declared with `package myPackage;` at the top of a file.
 - Packages map directly to your file-system directory structure (`java.awt.image` → folder `java/awt/image`).
 - Java looks for packages: (1) current working directory, (2) the `CLASSPATH` variable, (3) the `-classpath` flag.
 - Only members declared `public` are visible outside the package once imported.
-

Part 7: The `static` Keyword – Belonging to the Class, Not the Object

Concept

`static` members belong to the **class itself**, not to any individual object. There's only ever one copy, shared by everyone.

Why do we need it?

Some things don't make sense to duplicate per object — like a counter tracking "how many objects have been created so far," or utility methods like `Math.sqrt()` that don't need any object state.

Real-life Analogy

A school has one shared notice board (`static`), but each student has their own personal notebook (instance variable).

Java Example

```
class Counter {
    static int count = 0;        // shared across all objects
    Counter() { count++; }
}
Counter a = new Counter();
Counter b = new Counter();
// Counter.count is now 2
```

Key Points

- `static` members can be accessed **before any object exists** and without an object — that's why `main()` is static.
 - A static method can only directly access other static members. To touch an instance field, it needs an explicit object reference.
 - A static method **cannot** use `this` or `super` — there's no current object to refer to.
 - **Static initializer blocks** run exactly once, when the class is first loaded — great for one-time setup.
 - You **cannot** override a static method — it's resolved at *compile time* (early binding), not run time.
 - Only nested (inner) classes can be declared `static`.
-

Part 8: Polymorphism, Part A — Method Overloading

Concept

Overloading = multiple methods in the same class, same name, but different parameter lists.

Why do we need it?

Without it, you'd need awkward names like `add`, `addInt`, `addDouble`. Overloading lets one intuitive name handle many input types.

Java Example

```
class Calc {  
    void add(int a, int b)      { System.out.println(a + b); }  
    void add(double a, double b) { System.out.println(a + b); }  
}
```

Key Points

- Return type **alone** cannot distinguish two overloaded methods – the parameter list must differ.
 - Java will use **automatic type conversion** if no exact match exists (e.g., an `int` argument widens to `double` if only `add(double, double)` exists) – but an exact match always wins first.
 - This is called **compile-time polymorphism** – Java decides *which* version to call while compiling, not while running.
 - Constructors can be overloaded too (see Part 2).
-

Part 9: Pillar 3 – Inheritance

Concept

Inheritance lets one class (**subclass**) acquire the fields and methods of another (**superclass**), using `extends`.

Why do we need it?

Suppose you already built a `Vehicle` class. A `Car` and a `Bike` share a lot of that behavior. Instead of rewriting it, `Car extends Vehicle` and just adds what's unique.

Real-life Analogy

A `SportsCar` **is a** `Car`, which **is a** `Vehicle`. It inherits wheels and an engine, and adds a turbo boost of its own.

Java Example

```
class Vehicle {
    void start() { System.out.println("Engine started"); }
}
class Car extends Vehicle {
    void honk() { System.out.println("Beep beep!"); }
}
Car c = new Car();
c.start();    // inherited
c.honk();    // own method
```

Key Points

- Java supports only **single inheritance** for classes – one subclass, one direct superclass (no multiple inheritance).
- A subclass **cannot** access `private` members of its superclass directly.
- A superclass reference variable can point to a subclass object – but it can only access members defined by the superclass:

```
Vehicle v = new Car();    // v.honk() ❌ won't compile – Vehicle doesn't have honk()
```

It's the **type of the reference**, not the actual object, that decides what's visible at compile time.

Using `super`

`super` lets a subclass reach into its immediate superclass:

1. **Calling the superclass constructor:** `super(args);` – must be the *first* line in the subclass constructor.
2. **Accessing a hidden member:** `super.member` – used when the subclass has a field/method with the same name.

```
class BoxWeight extends Box {
    double weight;
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d);    // sets up the Box part first
        weight = m;        // then the new part
    }
}
```

 **Remember:** Constructors always finish in order of derivation – superclass first, subclass last. It makes sense: the subclass might depend on the superclass already being set up.

Common Beginner Mistakes

- Forgetting that `super()` must be the **first statement** in a constructor.
- Assuming a class can extend more than one class – it can't (that's what interfaces are for – see Part 12).

Part 10: `final` – Locking Things Down for Good

Concept

`final` has three different jobs depending on where you put it.

Applied to	Effect
A variable	Becomes a constant – must be initialized, can never be reassigned
A method	Cannot be overridden by any subclass
A class	Cannot be extended/inherited at all

Java Example

```
final int MAX_USERS = 100;           // constant
class Utility {
    final void log() { }              // can't be overridden
}
final class Config { }               // can't be subclassed at all
```

Key Points

- `final` guarantees immutability only for **primitives**. For a `final` reference-type variable, the reference itself can't change (it always points to the same object) – but the object's *internal* data can still change.
- A `final` method call can be resolved at **compile time** (early binding), which the compiler can optimize by *inlining* – a small performance win.
- A class marked `final` implicitly makes **all its methods final** too.

- You **cannot** mark a class both `abstract` and `final` — that's a contradiction (incomplete, yet unextendable).
-

Part 11: Polymorphism, Part B – Method Overriding & Dynamic Dispatch

Concept

Overriding happens when a subclass redefines a method it inherited, using the exact same name and parameter list.

Why do we need it?

Overloading picks a method at compile time based on parameter types. But sometimes you want the *object's actual type at run time* to decide behavior — that's overriding.

Real-life Analogy

Every `Shape` has a `draw()` method, but a `Circle` draws itself very differently from a `Square`. The caller just says "draw yourself" — the shape figures out how.

Java Example

```
class Shape { void draw() { System.out.println("Drawing shape"); } }
class Circle extends Shape { void draw() { System.out.println("Drawing ci

Shape s = new Circle();
s.draw();    // prints "Drawing circle" — decided at RUN TIME
```

Key Points

- This is called **run-time polymorphism** (a.k.a. **dynamic method dispatch**) — Java checks the *actual object type*, not the reference type, to decide which version runs.
- Overriding requires an **identical** method signature. If the signatures differ, it's just overloading, not overriding.
- When overriding, the access modifier in the subclass must be **equally or more open** than in the superclass (e.g., `protected` → `protected` or `public`, never `protected` → `private`).

- `final` methods **cannot** be overridden (see Part 10). Static methods also cannot be truly overridden — they're hidden, not polymorphic.
-

Part 12: Pillar 2 – Abstraction

Concept

Abstraction means exposing only the essential *what*, and hiding the messy *how*.

Why do we need it?

When you drive a car, you just need to know "press pedal → car moves." You don't need to know the combustion cycle happening under the hood. Abstraction lets code consumers use a class without knowing its internals.

Abstract Classes & Methods

```
abstract class Shape {
    abstract double area();           // no body — subclass's responsibility
    void describe() {                 // concrete method — has a body
        System.out.println("I am a shape");
    }
}

class Circle extends Shape {
    double radius = 5;
    double area() { return Math.PI * radius * radius; } // must implement
}
```

Key Points

- An `abstract` method has **no body** — the subclass *must* provide one.
- Any class with even one abstract method must itself be declared `abstract`.
- You **cannot** create an object of an abstract class directly — but you *can* create a reference of that type (this is exactly how run-time polymorphism works).
- You **cannot** have an abstract **constructor** or an abstract **static** method.
- Abstract classes *can* mix abstract and fully-implemented (concrete) methods.

Common Beginner Mistakes

- Trying to write `new Shape()` on an abstract class – always throws a compile error.
 - Forgetting to implement every abstract method in a subclass (unless that subclass is also declared `abstract`).
-

Part 13: Interfaces – A Pure Contract

Concept

An **interface** specifies *what* a class must do, without saying *how*. It's like an extreme version of an abstract class – historically, 100% abstract.

Why do we need it?

Java doesn't allow a class to extend more than one class (no multiple inheritance) – because if two parent classes did the same thing differently, the compiler wouldn't know which version to trust. Interfaces sidestep this: a class can **implement many** interfaces safely, because interfaces only describe *behavioral contracts*, not conflicting state.

Real-life Analogy

A "Flyable" interface is like a job description – it says "must be able to fly," but doesn't care whether you're a bird, a plane, or a drone.

Java Example

```
interface Flyable {
    void fly(); // implicitly public + abstract
    int MAX_ALTITUDE = 10000; // implicitly public, static, final

    default void land() { // JDK 8+: default method with a body
        System.out.println("Landing...");
    }
}

class Bird implements Flyable {
    public void fly() { System.out.println("Flap flap"); }
}
```

Key Points

- All interface methods are implicitly `public` and `abstract` (unless marked `default` or `static`).
- All interface variables are implicitly `public`, `static`, and `final` – essentially constants.
- A class can implement **multiple** interfaces; it can extend only **one** class.
- An interface can `extend` another interface (even multiple).
- **Default methods** (JDK 8+) let interfaces evolve without breaking every existing implementing class.
- **Static interface methods must have a body** and are **never inherited** by implementing classes or sub-interfaces.
- If a class implements two interfaces with the *same* default method and doesn't override it → compile error.
- The type of the **object**, not the reference, decides which overridden method runs – same dynamic dispatch rule as classes.

 **Interview Question:** *Why can't static interface methods be abstract?* Because static methods run on the interface itself (not on an instance), and Java resolves them at compile time. An empty (abstract) method called this way would have nothing to execute – so a body is mandatory.

Part 14: Abstract Class vs. Interface – Head to Head

Aspect	Abstract Class	Interface
Methods	Abstract + concrete	Only abstract (plus <code>default</code> / <code>static</code> since JDK 8)
Variables	final, non-final, static, non-static	Only <code>public static final</code>
Inheritance keyword	<code>extends</code>	<code>implements</code>
Multiple inheritance	Extends only one class, but can implement many interfaces	Can extend multiple interfaces
Access modifiers on members	Can use <code>private</code> , <code>protected</code> , etc.	Members are <code>public</code> by default
Can implement the other?	Can provide an interface's implementation	Cannot implement an abstract class

 **Rule of thumb:** Use an **abstract class** when subclasses share common code/state. Use an **interface** when you just need to guarantee a capability, possibly across unrelated classes.

Part 15: Abstraction vs. Encapsulation – Don't Mix Them Up

They're often confused because both "hide" something – but they hide *different* things, for *different* reasons.

Abstraction	Encapsulation
Hides unnecessary detail, shows only what's essential	Bundles code and data into one unit to protect it
Solves a problem at the design level	Solves a problem at the implementation level
Focused on the external view (what a user sees)	Focused on internal working (how it's protected)
Achieved using abstract classes and interfaces	Achieved using access modifiers (<code>private</code> , <code>getters</code> / <code>setters</code>)
The process of gaining information (deciding what to expose)	The process of containing information (locking it up)

 **Interview one-liner:** *Abstraction is about design – hiding complexity. Encapsulation is about implementation – hiding data.*

Part 16: The `finalize()` Method – Last Rites of an Object

Concept

`finalize()` is a method Java's garbage collector calls right before it reclaims an object's memory – a chance for cleanup.

Java Example

```
protected void finalize() {  
    // cleanup code, e.g., closing a file handle  
}
```

Key Points

- You never call `finalize()` yourself – the JVM calls it automatically, right before garbage collection reclaims the object.
 - Modern Java strongly discourages relying on `finalize()` (it's actually deprecated in recent versions) – prefer `try-with-resources` for cleanup. Still worth knowing conceptually and for legacy-code interviews.
-

Part 17: Enums – Named Constants, Leveled Up

Concept

An `enum` is a fixed list of named constants, but in Java it's secretly a **full class**.

Why do we need it?

Instead of magic numbers like `int status = 1;` (what does `1` even mean?), an enum gives meaningful names: `Status.ACTIVE`.

Java Example

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

Day today = Day.MON;
System.out.println(today.ordinal()); // 0 – position in the list
```

Key Points

- Declared with `enum`, and can live outside or inside a class (but never inside a method).
- Internally, each constant is a `public static final` object of the enum type.
- All enums implicitly extend `java.lang.Enum` – so an enum **cannot** extend anything else, and **cannot** be used as a superclass.
- An enum **can** implement interfaces.
- Built-in methods: `values()` (returns all constants), `ordinal()` (returns index/position), `valueOf(String)` (string → constant).
- An enum constructor is always `private` or default (never `public` / `protected`) – because you should never manually create more enum objects beyond the fixed list.
- Enums can only contain concrete methods – no abstract ones.
- Compare enum constants using `==` (safe and preferred) or `.equals()`.

The Big Picture: Cheat Sheet for Last-Minute Revision

Term	One-line memory hook
Class vs Object	Blueprint vs the actual house
Constructor	Auto-runs at birth, sets up initial state
<code>this</code>	"My own" field/object
Encapsulation	Data + methods in one box, access controlled
Access Modifiers	<code>private</code> < default < <code>protected</code> < <code>public</code> (restrictive → open)
<code>static</code>	Belongs to class, one shared copy
Overloading	Same name, different parameters — decided at compile time
Inheritance	<code>extends</code> , reuse + <code>super</code> to reach the parent
<code>final</code>	Constant / no-override / no-extend
Overriding	Same signature, subclass version wins — decided at run time (dynamic dispatch)
Abstraction	Abstract classes/interfaces — hide the <i>how</i>
Interface	Pure contract, multiple implementable, JDK8+ allows <code>default</code> / <code>static</code> bodies
<code>finalize()</code>	Cleanup right before garbage collection
Enum	A class-based fixed list of constants

 **Final Interview Tip:** If you're ever asked "What is OOP?" — don't recite definitions. Say: "OOP lets us model real-world entities as objects that bundle data and behavior (Encapsulation), expose only what's necessary (Abstraction), reuse code through parent-child relationships (Inheritance), and let one interface behave differently depending on context (Polymorphism)." That single sentence covers the whole guide.